



hexagone

École Supérieure d'Informatique

Notions fondamentales

4PYTH – Langage Python

Chris CHEVALIER



hexagone

École Supérieure d'Informatique

Sommaire

- #1** Mettre en place son environnement
- #2** Variables et types
- #3** Opérateurs et conditions
- #4** Structures de données
- #5** Boucles
- #6** Fonctions et modules
- #7** Erreurs et exceptions



#1

Mettre en place son environnement

Notions fondamentales



Python en quelques mots

- **Interprété** : pas de compilation, on exécute directement le code
- **Multiplateforme** : Windows, macOS, Linux
- **Lisible** : l'indentation fait partie de la syntaxe
- **Riche** : une bibliothèque standard fournie et des centaines de milliers de paquets sur [PyPI](#)

Créé par Guido van Rossum (première version en 1991), son nom est un clin d'œil aux Monty Python.

Python 2 ou Python 3 ?

Python 2 n'est plus maintenu depuis le 1er janvier 2020. Pourtant, beaucoup de vieux scripts sont encore écrits en Python 2. Il faut savoir les reconnaître et les adapter.



Reconnaître la version

```
print "pwned"      # Python 2  
print("pwned")     # Python 3
```



Installer Python

Installez Python 3.12 au minimum, idéalement la dernière version stable.

Windows

- Installateur de python.org
- Cochez « **Add python.exe to PATH** »
- Vérifiez : `py --version`
- ⚠ Si `python` ouvre le Microsoft Store : Paramètres → Applications → *Alias d'exécution d'applications* → désactivez `python.exe`

macOS

- Installateur de python.org ou `brew install python`
- Vérifiez : `python3 --version`
- ⚠ Le `python3` fourni avec les outils Xcode est **trop ancien** (3.9). Vérifiez toujours la version

Linux

- `sudo apt install python3 python3-venv python3-pip`
- Vérifiez : `python3 --version`
- ⚠ Sous Debian et Ubuntu, `venv` est un **paquet séparé**
- Kali : Python 3 est déjà installé



Exécuter du code Python

Mode interactif : pour tester une ligne

```
Terminal
$ python3                (py sous Windows)
>>> 2 ** 10
1024
>>> exit()
```

Script : un fichier `.py`, exécuté de haut en bas

```
Terminal
$ python3 scan.py        # macOS / Linux
> py scan.py             # Windows
```

Dans VS Code : installez l'extension Python, puis `Ctrl+Maj+P` → *Python: Select Interpreter* → choisissez celui du dossier `.venv`. Exécutez avec le bouton ► ou dans le terminal intégré.



Pourquoi un environnement virtuel ?

- Sans venv, tous les paquets s'installent **au même endroit**. Deux projets qui exigent deux versions d'une même bibliothèque entrent en conflit
- Un venv (*virtual environment*, environnement virtuel) est un dossier `.venv` **propre au projet**, avec son Python et ses paquets
- **Aucun droit administrateur** nécessaire
- **Reproductible** : la liste des paquets tient dans un fichier

Côté cyber

Chaque outil de sécurité s'installe dans son propre venv. Il n'y a pas de conflit entre outils et vous savez exactement ce qui est installé sur votre machine.

Attention au typosquatting

Des paquets malveillants imitent des noms connus à une lettre près (`requeusts` au lieu de `requests`). Vérifiez toujours le nom exact sur [PyPI](https://pypi.org) avant d'installer.



Créer et activer un venv

```
Windows (PowerShell)

cd mon-projet
py -m venv .venv
.venv\Scripts\Activate.ps1
```

```
macOS / Linux

cd mon-projet
python3 -m venv .venv
source .venv/bin/activate
```

- `python -m venv .venv` **crée** le venv, une seule fois par projet
- `activate` **active** le venv dans le terminal ouvert. Les commandes `python` et `pip` utilisent désormais le Python et les paquets du venv plutôt que ceux de l'ordinateur
- L'invite affiche `(.venv)`. C'est le signe que le venv est actif

PowerShell refuse l'activation (« l'exécution de scripts est désactivée sur ce système ») ? Une seule fois : `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`



Installer des paquets avec pip

```
Terminal (venv activé)

pip install requests      # installer un paquet
pip list                  # paquets installés
pip freeze > requirements.txt
pip install -r requirements.txt
deactivate                 # désactiver le venv
```

```
.gitignore

.venv/
__pycache__/
```

- **Activez toujours le venv avant `pip install`**, sinon le paquet s'installe sur tout l'ordinateur. L'activation vaut pour un terminal. À chaque nouveau terminal, on réactive. `deactivate` revient au Python de l'ordinateur
- `requirements.txt` liste les paquets et leurs versions. **On le versionne** dans Git
- `.venv` se recrée en une commande et `__pycache__` contient le code déjà compilé que Python garde pour démarrer plus vite. **On ne versionne ni l'un ni l'autre**



#2 Variables et types

Notions fondamentales



Les variables

Une variable est une **boîte étiquetée**. Le nom est l'étiquette, la valeur est le contenu. On la crée avec l'**opérateur d'affectation** `=`.

```
variables.py
username = "Chris"
failed_attempts = 0
MAX_ATTEMPTS = 3      # constante : majuscules

failed_attempts = failed_attempts + 1
print(username, failed_attempts)  # Chris 1
```

Règles de nommage (PEP 8)

- **snake_case** : `failed_attempts`
- En anglais, sans accent, explicite : `failed_attempts` plutôt que `fa`
- Sensible à la casse : `user` et `User` sont deux variables différentes
- Constantes en MAJUSCULES : `MAX_ATTEMPTS`

La **PEP 8** (*Python Enhancement Proposal* n° 8) est le guide officiel de style du code Python : nommage, indentation, longueur des lignes...



Les types de base

```
data_types.py

port = 443          # int
cvss_score = 9.8    # float
hostname = "srv-web-01" # str
is_admin = False    # bool
last_login = None    # aucune valeur

print(type(port))    # <class 'int'>
print(type(cvss_score)) # <class 'float'>
print(type(last_login)) # <class 'NoneType'>
```

Type	Contenu
int	entier
float	nombre à virgule (écrit avec un point)
str	chaîne de caractères, entre guillemets
bool	True ou False
None	absence de valeur

Typage dynamique : le type est porté par la valeur et déterminé à l'exécution, on ne le déclare pas ; `type()` permet de le vérifier. Le score CVSS note la gravité d'une vulnérabilité de 0 à 10.



Conversions, saisie et affichage

```
conversions.py

port = input("Port : ")      # on tape 8080
print(type(port))           # <class 'str'>

port = int(port)             # conversion en int
print(port + 1)              # 8081

score = float("7.5")         # 7.5
label = str(443)             # "443"
print("Port " + label)       # Port 443
```

`input()` renvoie toujours une chaîne de caractères, même si l'utilisateur tape un nombre. Pour calculer, il faut convertir avec `int()` ou `float()`.

- `print()` affiche une ou plusieurs valeurs, séparées par des espaces
- **Concaténation** : l'opérateur `+` colle deux chaînes de caractères bout à bout. Les deux côtés doivent être des chaînes. D'où le `str(443)`



Chaînes : indices et découpage

1	9	2	.	1	6	8	.	1	.	4	2
0	1	2	3	4	5	6	7	8	9	10	11
-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

```
indexes.py

ip = "192.168.1.42"

print(ip[0])      # 1
print(ip[-1])     # 2
print(ip[0:7])    # 192.168
print(ip[8:])     # 1.42
print(len(ip))    # 12
```

- Le premier indice est **0** ; les indices négatifs partent de la fin
- `s[début:fin]` : la **fin est exclue** ;
début omis = depuis le début, fin omise = jusqu'au bout
- Une chaîne **ne se modifie pas**.
`ip[0] = "2"` provoque une erreur



Méthodes de chaîne et f-strings

```
auth_log.py

line = "  Failed password for root from 10.0.0.5 "

line = line.strip()           # retire les espaces
words = line.split(" ")      # liste de mots
user, ip = words[3], words[-1]

print(line.startswith("Failed")) # True
print(user.upper())              # ROOT
print(f"Échec pour {user} depuis {ip}")
print(f"CVSS : {9.8123:.1f}")    # CVSS : 9.8
```

Méthode	Rôle
<code>strip()</code>	retire les espaces autour
<code>split(sep)</code>	découpe en liste
<code>lower()</code>	met en minuscules
<code>startswith(x)</code>	commence par <code>x</code> ?
<code>replace(a,b)</code>	remplace <code>a</code> par <code>b</code>
<code>x in s</code>	<code>s</code> contient <code>x</code> ?

La **f-string** (`f"...{variable}..."`) est une **concaténation moderne**. Au lieu de coller des morceaux avec `+`, on insère les valeurs directement dans le texte. `"Échec pour " + user` devient `f"Échec pour {user}"`, sans conversion avec `str()`.



Qu'affiche ce programme ?

```
quiz_1.py

user = "Admin"
ip = "10.0.0.5"
attempts = "3"

print(user.lower() + "@" + ip[-1])
print(attempts * 2)
print(int(attempts) * 2)
```

`admin@5` : `+` concatène les chaînes, `ip[-1]` est le dernier caractère

`33` : `attempts` est une chaîne, `* 2` la répète

`6` : après conversion en entier, `*` `2` multiplie



#3 Opérateurs et conditions

Notions fondamentales



Les opérateurs

```
operators.py

print(17 + 5, 17 - 5, 17 * 5)  # 22 12 85
print(17 / 5)                  # 3.4   division
print(17 // 5)                 # 3     division entière
print(17 % 5)                  # 2     reste (modulo)
print(2 ** 10)                 # 1024  puissance

port = 8080
print(port == 443, port != 443, port > 1024)
# False True True
```

Comparaison	Signification
<code>== / !=</code>	égal / différent
<code>< / <=</code>	inférieur / ou égal
<code>> / >=</code>	supérieur / ou égal

% (modulo) donne le reste d'une division. Il permet de « faire le tour » d'un cycle (les heures d'une horloge, les lettres d'un alphabet). Il est très utilisé en cryptographie.

Ne pas confondre : `=` affecte une valeur, `==` compare deux valeurs. L'opérateur `===` de JavaScript ou de PHP **n'existe pas** en Python.



if, elif, else

```
access_control.py

role = "analyst"
failed_attempts = 1

if failed_attempts >= 3:
    print("Compte verrouillé")
elif role == "admin":
    print("Accès total")
elif role == "analyst":
    print("Accès en lecture")    # ← exécuté
else:
    print("Accès refusé")
```

- Les conditions sont testées **dans l'ordre**. Seul le premier bloc dont la condition est vraie s'exécute
- `elif` et `else` sont facultatifs
- Les deux-points `:` ouvrent un bloc ; l'**indentation (4 espaces)** délimite ce bloc
- Une erreur d'indentation change le sens du programme, voire l'empêche de s'exécuter



and, or, not, in

```
login_check.py

password_ok = True
mfa_ok = False
blocked_ips = ["10.0.0.5", "203.0.113.7"]
ip = "10.0.0.5"

if password_ok and mfa_ok:
    print("Connexion autorisée")
if not mfa_ok:
    print("Code MFA manquant")           # ← exécuté
if ip in blocked_ips or ip.startswith("192.0.2."):
    print(f"{ip} est bloquée")          # ← exécuté
```

Opérateur	Vrai si...
a and b	les deux sont vrais
a or b	au moins un est vrai
not a	a est faux
x in s	x appartient à s (liste, chaîne...)



match / case

```
commands.py

command = "scan"

match command:
    case "scan":
        print("Lancement du scan...")    # ← exécuté
    case "report" | "export":
        print("Génération du rapport")
    case "quit":
        print("Au revoir")
    case _:
        print("Commande inconnue")
```

- Compare une valeur à plusieurs **motifs**, du premier au dernier
- `|` signifie « ou »
- `_` est le cas par défaut
- Disponible depuis Python 3.10. C'est l'équivalent du bloc `switch / case` de nombreux langages (C, Java, JavaScript...)
- Idéal pour un **menu**



#4 Structures de données

Notions fondamentales



Les listes

```
ports.py

open_ports = [443, 22, 80]

print(open_ports[0])      # 443
open_ports.append(3306)   # ajoute à la fin
open_ports.remove(80)     # retire la valeur 80
open_ports.sort()         # trie sur place
print(open_ports)         # [22, 443, 3306]
print(len(open_ports))    # 3
print(22 in open_ports)   # True
print(open_ports[-2:])    # [443, 3306]
```

- Une collection **ordonnée** et **modifiable**, entre crochets `[]`
- Indices et découpage fonctionnent **comme pour les chaînes de caractères**. `liste[-2:]` part du 2e élément en partant de la fin et va jusqu'au bout. `logs[-10:]` donne ainsi les 10 dernières lignes d'un journal
- `remove()` retire la **première** occurrence trouvée
- Toutes les méthodes : [documentation officielle](#)



Tuples et ensembles

```
targets.py

# Tuple : non modifiable
target = ("10.0.0.5", 22)
ip, port = target          # déballage
print(f"{ip}:{port}")      # 10.0.0.5:22

# Ensemble : valeurs uniques, sans ordre
ips = ["10.0.0.5", "10.0.0.9", "10.0.0.5"]
unique_ips = set(ips)
print(len(unique_ips))     # 2
unique_ips.add("10.0.0.12")
print("10.0.0.9" in unique_ips)  # True
```

Tuple ()

Ordonné, **non modifiable** : parfait pour une valeur composée qui ne doit pas changer (adresse et port)

Ensemble set

Sans doublon et sans ordre : parfait pour dédupliquer les IP d'un journal



Les dictionnaires

```
account.py

user = {"username": "Chris", "role": "analyst",
        "mfa": True}

print(user["role"])           # analyst
user["role"] = "admin"        # modification
user["last_ip"] = "10.0.0.5"  # ajout d'une clé
print(user.get("email"))      # None
print(user.get("email", "?")) # ?
print("mfa" in user)          # True
print(list(user.keys()))
# ['username', 'role', 'mfa', 'last_ip']
```

- Des associations **clé** → **valeur**, entre accolades `{}`
- On accède par la **clé**, pas par un indice
- Les clés sont **uniques**
- `user["email"]` provoque une erreur `KeyError` si la clé n'existe pas ; `user.get("email")` renvoie `None`



Combiner et choisir

```
accounts.py

accounts = [
    {"username": "Chris", "role": "admin"},
    {"username": "Manon", "role": "analyst"},
]
new_account = {"username": "Cosmo", "role": "guest"}
accounts.append(new_account)

print(len(accounts))          # 3
print(accounts[1]["username"]) # Manon
```

	Ordre	Modifiable	Doublons
list	oui	oui	oui
tuple	oui	non	oui
set	non	oui	non
dict	oui	oui	clés : non

Une **liste de dictionnaires** représente une table d'enregistrements, comme les lignes d'un tableur ou d'une base de données.



#5 Boucles

Notions fondamentales





La boucle for

```
for.py

open_ports = [22, 80, 443]
for port in open_ports:
    print(f"Port {port} ouvert")

for i in range(1, 4):    # 1, 2, 3
    print(f"Tentative {i}")
```

```
for_advanced.py

for i, port in enumerate(open_ports, 1):
    print(i, port)    # 1 22 / 2 80 / 3 443

user = {"name": "Chris", "role": "admin"}
for key, value in user.items():
    print(f"{key} = {value}")
```

- `for` parcourt **chaque élément** d'une séquence : liste, chaîne, tuple, ensemble, dictionnaire
- `range(début, fin)` produit une suite d'entiers, **fin exclude** ; `enumerate()` numérote les éléments ; `.items()` donne clé et valeur



while, break, continue

```
login.py

SECRET = "S3cur3!"
attempts = 0

while True:
    password = input("Mot de passe : ")
    attempts += 1      # attempts = attempts + 1
    if password == SECRET:
        print("Accès autorisé")
        break          # sortie immédiate
    if attempts == 3:
        print("Compte verrouillé")
        break
```

- `while cond:` répète **tant que** la condition est vraie
- `while True:` sans fin, sauf `break`
- `continue` passe au tour suivant
- ⚠ **Boucle infinie** : prévoyez toujours une sortie (Ctrl + C pour l'interrompre)

```
continue.py

for port in [21, 22, 80]:
    if port == 21:
        continue      # saute FTP
    print(port)        # 22 puis 80
```



Compter avec un dictionnaire

```
failures.py

log = [
    "Failed password for root from 10.0.0.5",
    "Accepted password for Chris from 10.0.0.9",
    "Failed password for admin from 10.0.0.5",
]
failures = {}
for line in log:
    if line.startswith("Failed"):
        ip = line.split(" ")[-1]
        failures[ip] = failures.get(ip, 0) + 1
print(failures)           # {'10.0.0.5': 2}
```

- `failures.get(ip, 0)` renvoie le compteur actuel ou `0` si l'IP est nouvelle
- On ajoute 1 et on enregistre le résultat sous la même clé

Détection de brute force : compter les échecs par source, puis alerter au-delà d'un seuil. C'est ce que font fail2ban (blocage des IP insistantes) et les **SIEM** (*Security Information and Event Management*), outils centraux d'un SOC.



Qu'affiche ce programme ?

```
quiz_2.py
ports = [21, 22, 80, 443, 8080]
count = 0
for port in ports:
    if port < 80:
        continue
    if port == 443:
        break
    count += 1
print(count)

total = 0
for i in range(3):
    total += i
print(total)
```

1 : 21 et 22 sont sautés par `continue`, 80 est compté, puis la boucle s'arrête sur 443. Le port 8080 n'est jamais atteint

3 : `range(3)` produit 0, 1 et 2, donc $0 + 1 + 2 = 3$



#6 Fonctions et modules

Notions fondamentales



Définir une fonction

```
checks.py

def is_valid_port(port):
    """Vrai si port est un port TCP valide."""
    return 0 <= port <= 65535

def format_alert(ip, level="INFO"):
    return f"[{level}] Activité suspecte : {ip}"

print(is_valid_port(8080))      # True
print(is_valid_port(70000))     # False
print(format_alert("10.0.0.5")) # [INFO] ...
print(format_alert("10.0.0.5", level="HIGH"))
# [HIGH] Activité suspecte : 10.0.0.5
```

- `def nom(paramètres):` définit un bloc réutilisable
- `return` **renvoie** un résultat et termine la fonction. Il n'est pas obligatoire. Sans `return`, la fonction renvoie `None` (on parle alors parfois de **procédure**)
- La **docstring** (`"""..."""`) documente la fonction
- `level="INFO"` : valeur **par défaut**, utilisée si l'argument est omis



Les bonnes pratiques

Ne pas se répéter

Un même code copié à plusieurs endroits devient une fonction. On le corrige alors à un seul endroit (principe **DRY**, *Don't Repeat Yourself*).

Une fonction, une tâche

Des fonctions courtes, **15 lignes au maximum**. Plus longue, elle fait plusieurs choses. Découpez-la.

Rester générique

Passez les valeurs en **paramètres** plutôt que de les écrire en dur.

```
is_valid_port(port)
```

sert pour n'importe quel port.

Bien nommer : un verbe pour une fonction (`check_password` , `count_failures`), comme pour les variables et les fichiers. En anglais, en `snake_case` et explicite : on doit comprendre ce que fait la fonction sans lire son code.



Portée et retours multiples

```
parse.py

def parse_login(line):
    words = line.split(" ")    # locale
    return words[3], words[-1] # un tuple

log = "Failed password for root from 10.0.0.5"
user, ip = parse_login(log)
print(user, ip)    # root 10.0.0.5
print(words)       # erreur !
```

```
Terminal

NameError: name 'words' is not defined
```

- Variable créée dans une fonction : **locale**, elle n'existe que dans la fonction et disparaît à sa sortie
- Variable créée hors de toute fonction : **globale**, à éviter dans les fonctions (passez-la en paramètre)
- Un `if` ou une boucle **ne crée pas de portée**. Une variable créée dans un bloc reste accessible après lui. Seules les fonctions et les classes en créent une
- Pour renvoyer plusieurs valeurs : un **tuple**, que l'on déballe



Importer un module

```
security.py

"""Utilitaires de sécurité."""

def is_valid_port(port):
    return 0 <= port <= 65535

if __name__ == "__main__":
    # si on exécute ce fichier directement
    print(is_valid_port(443))
```

```
main.py

import security
from security import is_valid_port

print(security.is_valid_port(22)) # True
print(is_valid_port(-1))          # False
```

- Tout fichier `.py` est un **module** importable depuis un fichier du même dossier
- `if __name__ == "__main__":` contient le code exécuté **seulement** quand on lance le fichier directement, pas quand on l'importe



La boîte à outils de la bibliothèque standard

Module	Usage en cybersécurité
<code>os</code> , <code>pathlib</code>	parcourir des dossiers, manipuler des chemins
<code>sys</code> , <code>argparse</code>	lire les arguments d'un outil en ligne de commande
<code>re</code>	rechercher des motifs dans des journaux (expressions régulières)
<code>datetime</code>	horodater, comparer des dates d'événements
<code>secrets</code>	générer mots de passe et jetons
<code>hashlib</code> , <code>json</code>	calculer des empreintes (hash), sauvegarder des données
<code>socket</code>	communiquer sur le réseau

Référence complète : [la bibliothèque standard de Python](#).



Zoom : random ou secrets ?

Deux modules de la bibliothèque standard produisent du hasard... mais un seul convient à la sécurité.

✗ random : prévisible

```
random_demo.py

import random

random.seed(2026)      # même graine...
a = random.randint(0, 999999)
random.seed(2026)
b = random.randint(0, 999999)
print(a, b)            # 124906 124906
print(a == b)          # True
```

✓ secrets : conçu pour la sécurité

```
secrets_demo.py

import secrets
import string

token = secrets.token_hex(16) # jeton
print(token)                  # 32 caractères au hasard

c = secrets.choice(string.ascii_letters)
print(c)                       # une lettre au hasard
```

`random` est **pseudo-aléatoire**. Qui connaît son état prédit la suite. Parfait pour un jeu, **jamais** pour un mot de passe, un jeton ou une clé. Utilisez `secrets`.



#7 Erreurs et exceptions

Notions fondamentales



Lire un traceback

```
Terminal : python scan.py (on tape « https »)

Traceback (most recent call last):
  File "scan.py", line 1, in <module>
    port = int(input("Port : "))
ValueError: invalid literal for int() with base 10: 'https'
```

- Lisez **de bas en haut** : le type d'erreur et son message
- Puis le **fichier**, la **ligne** et le code fautif
- Savoir lire un traceback, c'est savoir **se débloquent seul**

Erreur	Cause fréquente
SyntaxError	faute de syntaxe, : oublié
NameError	variable inconnue
TypeError	"3" + 1
ValueError	int("https")
IndexError	indice inexistant
KeyError	clé absente



try, except, else, raise

```
ask_port.py

def ask_port():
    while True:
        try:
            port = int(input("Port : "))
        except ValueError:
            print("Entrez un nombre entier.")
        else:
            if 0 <= port <= 65535:
                return port
            raise ValueError("port hors limites")

print(f"Scan du port {ask_port()}")
```

- `try` : le code qui peut échouer
- `except ValueError` : ce qu'on fait **si cette erreur précise** survient ; le programme ne plante plus. C'est l'équivalent du `catch` d'autres langages (Java, JavaScript, PHP)
- `else` : exécuté **seulement si** aucune erreur n'a eu lieu
- `raise` : **lever** soi-même une erreur quand une valeur est inacceptable. C'est l'équivalent du `throw` d'autres langages



Les exceptions, côté sécurité

✗ À éviter

```
silence.py

try:
    port = int(user_input)
except:
    pass      # erreur effacée sans trace
```

✓ À préférer

```
precise.py

try:
    port = int(user_input)
except ValueError:
    print("Port invalide.") # sobre
```

- Un `except: pass` **masque les anomalies**, y compris une tentative d'attaque. On attrape toujours une erreur **précise**
- N'affichez jamais un traceback complet à l'utilisateur. Chemins, versions et noms de fichiers sont une **fuite d'informations** précieuse pour un attaquant
- Le détail de l'erreur se consigne dans un **journal**, côté serveur



Cet après-midi

Horaire	Programme
14h00 – 15h15	Défis 1 à 3 : politique de mots de passe · détection de brute force dans un journal · générateur de mots de passe avec secrets
15h15 – 16h45	TP1 – Fondation : gestion fonctionnelle de l'équipage
16h45 – 17h00	Kahoot : 10 questions sur la journée

Les défis sont **notés en séance**. Appelez-moi pour me montrer chaque défi réussi.



Des questions ?

Chris CHEVALIER

chris.chevalier@ecole-hexagone.com